

CLAUDE CODE DEEP DIVE WORKSHOP

Chapter 2 - Agents (Subagents)

맞춤 서브에이전트로 작업을 분담하고 병렬화하기

Update : 2026.09 / Claude Code v2.1 최신 채널 기준, 공식 문서 code.claude.com/docs 반영

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com

Chapter 2 Agenda

9 Parts / 120 Slides

PART 01	Subagents 개념과 원리	격리 컨텍스트, Built-in, 실행 모델 / 17p
PART 02	Subagent 정의 방법	스코프 5계층, 프론트매터 15필드 / 19p
PART 03	Agent 도구와 디스패치	자동 위임, @멘션, fork, 중첩, resume / 17p
PART 04	Pattern 1 / Code Reviewer	리뷰 에이전트와 영속 메모리 / 12p
PART 05	Pattern 2 / Tester	테스트 작성과 worktree 격리 / 10p
PART 06	Pattern 3 / Security Scanner	읽기 전용 스캐너와 훅 검증 / 10p
PART 07	Pattern 4 / Docs Writer	문서 에이전트와 skills 프리로드 / 9p
PART 08	Pattern 5 / Migration Bot	대규모 마이그레이션 안전 장치 / 9p
PART 09	Recap & Labs	요약, FAQ, 실습 4종, Q&A / 13p

Chapter 2 Learning Objectives

본 챕터에서 달성할 학습 목표

AT THE END OF THIS CHAPTER

서브에이전트의 격리 모델을 설명하고, 프론트매터로 맞춤 에이전트를 정의하며, 자동 위임과 명시 호출, 병렬 실행을 실무 워크플로에 적용할 수 있습니다.

OBJ 1

격리 모델 이해

컨텍스트 격리와 요약 회수, fork와 팀의 경계를 설명.

OBJ 2

에이전트 정의

프론트매터 15필드로 도구, 모델, 메모리, hooks 설계.

OBJ 3

디스패치 운용

자동 위임, @멘션, 병렬과 체이닝, resume을 활용.

OBJ 4

실전 패턴 구축

리뷰어부터 마이그레이션 봇까지 5종을 직접 구현.

9 Parts

구조화된 학습 경로

4 Labs

Chapter 2 실습 랩

5 Patterns

실전 에이전트 구현

~3h + 1.5h

강의와 실습 시간

02

Agents (Subagents)

위임과 병렬화로 컨텍스트를 지키는 기술

CHAPTER 02 / PART 01

Subagents 개념과 원리

격리 컨텍스트와 위임의 가치

1. Subagent 정의와 컨텍스트 격리 모델
2. Built-in 에이전트: Explore, Plan, general-purpose
3. Foreground와 Background 실행 모델
4. Fork, Agent Teams, Background agents와의 경계

Subagent란 무엇인가

자기만의 컨텍스트 윈도우를 가진 격리 인스턴스

DEFINITION

Subagent는 특정 작업을 처리하는 격리된 Claude 인스턴스입니다.

자체 컨텍스트 윈도우, 맞춤 시스템 프롬프트, 별도 도구 권한으로 독립 작업 후 요약만 돌려줍니다.

ISOLATED

자체 컨텍스트

메인 대화와 분리된 윈도우
에서 작업, 상호 오염 없음.

FOCUSED

전용 프롬프트

역할에 특화된 시스템 프롬프트로 한 가지 일에 집중.

SCOPED

도구 격리

허용 도구를 좁혀 읽기 전용
등 안전한 권한 설계.

SUMMARY

요약 회수

수십 개 도구 호출의 결과가
요약 한 덩이로 귀환.

브라우저 탭

겉가지 추적의 비유

1 Session

한 세션 안의 워크

Markdown

정의 파일 형식

Agent tool

생성 도구 이름

왜 위임하는가

컨텍스트 오염이 품질을 깎는다

THE PROBLEM

검색 결과, 로그, 파일 내용이 메인 대화를 채우면 모델의 주의가 분산되고 응답 품질이 떨어집니다.
다시 참조하지 않을 출력이 특히 낭비입니다.

위임 없이 (메인에서 직접)

1. 30개 파일 읽기가 그대로 이력에 축적
2. 테스트 로그 수천 줄이 윈도우 점유
3. Compaction이 빨리 오고 초기 지시 소실
4. 긴 세션일수록 응답 품질 하락

위임하면 (Subagent 경우)

1. 대량 읽기는 자식 컨텍스트에서 소화
2. 메인에는 실패 테스트 요약만 도착
3. 본 대화는 결정과 방향에만 사용
4. 긴 세션에도 컨텍스트가 가볍게 유지

Main Agent vs Subagent

역할과 특성 비교

컨텍스트	메인: 전체 대화 이력 보유	자식: 위임 메시지로 새 출발
시스템 프롬프트	메인: Claude Code 전체 프롬프트	자식: 정의 파일 본문 + 환경 정보
도구	메인: 세션의 전체 도구	자식: tools 필드로 좁힌 집합
산출물	메인: 사용자와의 대화	자식: 요약 결과 한 덩이
수명	메인: 세션과 함께	자식: 작업 완료 시 종료, resume 가능

격리의 실체 / 시작 시 무엇이 로드되나

Subagent 초기 컨텍스트의 다섯 구성

System Prompt

에이전트 자신의 프롬프트 + 작업 디렉토리 등 환경 정보, Claude Code 전체 프롬프트는 미포함

Task Message

메인 Claude가 작성한 위임 지시문, 대화 이력은 오지 않음

CLAUDE.md + Memory

메인이 로드한 메모리 계층 전체, 단 Explore와 Plan은 생략

Git Status

부모 세션 시작 시점 스냅샷, Explore와 Plan은 생략

Preloaded Skills

skills 필드에 지정한 스킬의 본문 전체

위임의 5대 가치

공식 문서가 정리한 Subagent의 효용

VALUE 1

Preserve Context

탐색과 구현 출력을 메인 대화 밖에 격리.

VALUE 2

Enforce Constraints

도구 제한으로 읽기 전용 등 안전선 강제.

VALUE 3

Reuse Configs

사용자 레벨 정의로 전 프로젝트 재사용.

VALUE 4

Specialize

도메인 특화 프롬프트로 성공률 상승.

VALUE 5

Control Costs

haiku 같은 경량 모델로 라우팅해 비용 절감.

+ TEAM

팀 자산화

프로젝트 정의를 버전 관리로 팀과 공유, 공동 개선.

Built-in / Explore

읽기 전용 탐색 전문 에이전트

EXPLORE AGENT

코드베이스 검색과 분석에 최적화된 읽기 전용 에이전트입니다.

Claude가 수정 없이 코드를 이해해야 할 때 자동으로 위임합니다.

MODEL

메인 모델 상속

v2.1.198부터 상속,
Claude API에서는 Opus
상한 적용.

TOOLS

읽기 전용

Write와 Edit 거부, 검색과
읽기 도구만 허용.

DEPTH

3단계 강도

quick, medium, very
thorough를 호출 시 지정.

LIGHT

경량 설계

CLAUDE.md와 git 상태를
생략해 빠르고 저렴.

상한 Opus

Claude API 캡

직접 상속

Bedrock 등 타 공급자

model: haiku

동명 정의로 오버라이드

One-shot

resume 불가 특성

Built-in / Plan, general-purpose, 기타

나머지 내장 에이전트 카탈로그

Plan	Plan 모드의 조사 담당, 읽기 전용, 모델 상속	탐색 출력을 별도 창에 격리
general-purpose	전체 도구, 모델 상속, 탐색과 수정 병행	복잡한 다단계 작업
statusline-setup	Sonnet 고정	/statusline 실행 시 자동
claude-code-guide	Haiku 고정	Claude Code 기능 질문 시 자동
비활성화	permissions.deny에 Agent(Explore) 등	Agent 도구 자체 deny도 가능

4+

내장 에이전트 수

One-shot

Explore, Plan 특성

DISABLE_

EXPLORE_PLAN_AGENTS=1

SDK

BUILTIN_AGENTS 제거 변수

실행 모델 / Foreground vs Background

v2.1.198부터 백그라운드가 기본값

BACKGROUND BY DEFAULT

이제 서브에이전트는 기본적으로 백그라운드에서 둡니다.

결과가 즉시 필요할 때만 포그라운드로 실행되며, 권한 프롬프트는 메인 세션에 떠오릅니다.

FG

Foreground

완료까지 메인 차단, 결과가 다음 판단에 즉시 필요할 때.

BG

Background

동시 진행, 완료 시 결과가 메시지로 도착. 현재 기본값.

PERMS

권한 표면화

백그라운드의 승인 요청이 메인에 뜨고 요청 에이전트 표기.

CONTROL

조작

Ctrl+B로 실행 중 작업을 백그라운드 전환, 요청으로 지정 가능.

v2.1.198+

배경 기본화 버전

Ctrl+B

백그라운드 전환 키

Esc

개별 승인 거부

DISABLE_BACKGROUND

_TASKS=1 전체 비활성

상태 모델 / 종료 후에도 살아있다

트랜스크립트와 재개 가능성

LIFECYCLE

각 호출은 새 인스턴스로 시작하지만, 완료된 에이전트의 대화 기록은 파일로 남아 이어서 재개할 수 있습니다.

Explore와 Plan만 일회성입니다.

FRESH

새 출발

호출마다 새 인스턴스, 이전 호출의 대화는 자동 계승 없음.

ID

Agent ID

완료 시 메인 ID를 수신, 이 ID로 특정 인스턴스 지목.

RESUME

재개

이어서 해달라고 하면 전체 이력을 가진 채 그 지점부터 계속.

FILE

트랜스크립트

세션 폴더의 subagents/ 아래 JSONL, 30일 자동 정리.

One-shot

Explore, Plan 예외

JSONL

agent-{id} 파일

30일

cleanupPeriodDays 기본

독립

메인 압축과 무관

모델 해석 우선순위

Subagent가 어떤 모델로 도는가

1 환경변수

CLAUDE_CODE_SUBAGENT_MODEL, alias나 모델 ID 지정 시 최우선

2 호출 파라미터

메인 Claude가 Agent 도구 호출에 함께 넘기는 model 값

3 Frontmatter

정의 파일의 model 필드: sonnet, opus, haiku, fable, 전체 ID, inherit

4 메인 모델

위가 모두 없으면 메인 대화의 모델을 그대로 상속 (기본값)

경계 지도 / 4가지 병렬화 수단

Subagent, Fork, Background agents, Agent Teams

Subagent

한 세션 안의 워커, 새 컨텍스트, 요약 회수

결가지 격리, 병렬 리서치

Fork

대화 전체를 물려받은 서브에이전트

설명 없이 말기는 결가지

Background agents

독립 세션 여러 개를 한 화면에서 관찰

무관한 작업들의 병렬 운영

Agent Teams

세션들끼리 메시지로 협업, 더 무겁고 비쌈

지속 병렬, 컨텍스트 초과 작업

판단 기준

소통 필요 없으면 Subagent, 필요하면 Teams

규모와 결합도가 기준

Subagent 6대 사용 사례

위임이 가장 큰 가치를 내는 순간

CASE 1

고볼륨 격리

테스트 스위트 실행, 로그 처리의 대량 출력 격리.

CASE 2

병렬 리서치

독립 모듈 여러 개를 동시에 조사, 결과만 종합.

CASE 3

제2의 시선

구현과 분리된 신선한 컨텍스트로 리뷰와 검증.

CASE 4

권한 강제

읽기 전용, 도메인 한정 등 도구 제약을 역할로 고정.

CASE 5

문서 조회

외부 문서 다량 폐치를 지식에서 소화, 요약만 회수.

CASE 6

반복 역할

같은 지시를 반복하는 워커를 정의 파일로 상비군화.

Cost & Latency 고려사항

위임에도 가격표가 있다

OVERHEAD AWARENESS

서브에이전트는 새 컨텍스트에서 출발하므로 상황 파악 시간이 들고, 결과 회수도 토큰을 씁니다.

이득이 오버헤드를 넘는지 판단이 필요합니다.

STARTUP

기동 비용

새 출발이라 컨텍스트 수집 시간 발생, 빠른 단발 작업엔 비효율.

RETURN

회수 비용

다수 에이전트의 상세 결과가 메인을 다시 채울 수 있음.

MODEL

모델 배분

탐색성 워커는 haiku 지정으로 비용 절감.

CACHE

캐시 관점

fork는 메인의 프롬프트 캐시를 공유해 신규 스폰보다 저렴.

Fresh start

기동 오버헤드 원인

요약 지시

회수 비용 통제법

haiku

탐색 워커 권장

Cache hit

fork의 비용 이점

Subagent 안티 패턴

위임하지 말아야 할 순간

피해야 할 것

1. 잦은 왕복이 필요한 대화형 작업 위임
2. 계획, 구현, 테스트가 맥락을 공유하는 일 분리
3. 한 줄 수정 같은 초단발 작업 위임
4. 전 단계 결과에 의존하는 조사들의 병렬화
5. 만능 에이전트 하나로 모든 역할 해결

대신 이렇게

1. 왕복형 작업은 메인 대화에서 직접
2. 맥락 공유 단계는 한 흐름으로 유지
3. 빠른 확인은 /btw, 맥락 질문에 적합
4. 의존 조사는 체이닝으로 순차 실행
5. 역할별 단일 책임 에이전트로 분리

Part 1 Recap

개념과 원리 핵심 정리

KEY TAKEAWAY

Subagent는 자체 컨텍스트에서 일하고 요약만 돌려주는 격리 워커입니다.

이제 기본이 백그라운드 실행이며, 소통이 필요한 규모는 Agent Teams의 영역입니다.

핵심 정리

1. 정의: 격리 컨텍스트 + 요약 회수
2. 로드: 프롬프트, 지시문, 메모리, git
3. 내장: Explore, Plan, general-purpose
4. 실행: 백그라운드 기본, Ctrl+B
5. 경계: fork, BG agents, Teams

다음 파트

1. Part 2 정의 방법으로 이동
2. 스코프 5계층과 우선순위
3. 프런트매터 15개 필드 완전 정복
4. memory, skills, mcpServers
5. 공식 예시 해부

CHAPTER 02 / PART 02

Subagent 정의 방법

Markdown 한 장으로 워커를 상비군화

1. 정의 파일 해부와 스코프 5계층
2. 필수 필드: name, description 작성법
3. 도구, 모델, 권한 모드, 메모리 필드
4. skills 프리로드와 mcpServers 스코프

정의 파일 해부

YAML Frontmatter + 시스템 프롬프트 본문

.claude/agents/code-reviewer.md

```
---  
name: code-reviewer  
description: Reviews code for quality and best practices  
tools: Read, Glob, Grep  
model: sonnet  
---
```

You are a code reviewer. When invoked, analyze the code and provide specific, actionable feedback on quality, security, and best practices.

Frontmatter
메타데이터와 설정

본문
시스템 프롬프트

필수 2
name, description

파일명 자유
정체성은 name 필드

스코프 5계층과 우선순위

같은 이름이 충돌하면 위가 이긴다

1 Managed

조직 관리 설정 디렉토리의 `.claude/agents/`, 관리자가 배포

2 `--agents` 플러그

실행 시 JSON으로 주입, 세션 한정, 디스크 저장 없음

3 Project

`.claude/agents/`, 버전 관리로 팀 공유, 실무의 중심

4 User

`~/.claude/agents/`, 내 모든 프로젝트에서 사용

5 Plugin

플러그인의 `agents/`, 스코프 이름 `my-plugin:name`으로 등록

배치 규칙 상세

재귀 스캔, 이름 유일성, 모노레포

PLACEMENT RULES

agents 디렉토리는 재귀 스캔되어 하위 폴더로 정리할 수 있습니다.

정체성은 오직 name 필드이므로 이름 유일성이 관리 포인트입니다.

RECURSE

하위 폴더 정리

agents/review/,
agents/research/ 같은
분류 폴더 가능.

UNIQUE

이름 유일성

동일 스코프 중복 name은
하나만 로드, /doctor가 중
복 보고.

NESTED

모노레포 규칙

중첩 .claude 간 동명 충돌
시 작업 위치에 가까운 정의
승리.

ADD-DIR

추가 디렉토리

--add-dir로 붙인 경로의
.claude/agents/도 함께
로드.

name만

정체성 결정 요소

/doctor

중복 감지 v2.1.196+

Closest wins

중첩 규칙 v2.1.178+

Plugin 예외

하위 폴더가 스코프 ID

파일 워치 / 재시작이 필요 없다

저장하면 몇 초 안에 반영

LIVE RELOAD

Claude Code가 agents 디렉토리를 감시합니다.

파일을 추가하거나 수정하면 몇 초 안에 감지되어 다음 위임부터 새 정의가 적용됩니다.

WATCH

감시 대상

~/.claude/agents/와
.claude/agents/ 양쪽 모두.

APPLY

적용 시점

다음 위임부터 갱신된 정의
사용, 세션 재시작 불필요.

EXCEPT 1

첫 생성 예외

세션 시작 시 없던 agents
폴더를 새로 만들면 재시작
필요.

EXCEPT 2

옵션 예외

--disable-slash-
commands 세션은 감시 자
체가 꺼짐.

수 초

감지 지연

No restart

일반적인 경우

새 폴더

재시작 예외 1

SDK 주의

startup 로드 방식

생성 방법 / Claude에게 시키기

v2.1.198부터 /agents 위저드는 제거

~/proj \$ claude

> ~/.claude/agents/ 에 code-improver 서브에이전트를 만들어줘. 파일을 스캔해서 가독성, 성능, 모범 사례 개선점을 제안하는 역할이야. 각 이슈마다 문제 설명, 현재 코드, 개선 코드를 보여줘. 읽기 전용으로 하고 모델은 sonnet을 써.

Claude가 name, description, tools, model,
시스템 프롬프트를 갖춘 파일을 작성

> code-improver 에이전트로 이 프로젝트 개선점 제안해줘
저장 후 몇 초 뒤 바로 위임 가능

자연어

요구사항 전달

/agents

위저드 제거 v2.1.198

직접 편집

파일 수작성 병행

즉시 시험

생성 후 바로 호출

name과 description

자동 위임을 좌우하는 필수 2필드

THE TRIGGER FIELDS

Claude는 description을 읽고 위임 여부를 판단합니다.

언제 이 에이전트를 써야 하는지가 명확할수록 자동 위임의 정확도가 올라갑니다.

NAME

소문자와 하이픈

고유 식별자, 혹에는
agent_type으로 전달됨.

WHEN

시점 서술

무엇을 하는지보다 언제 쓰는
지를 담아야 위임 판단 가능.

PROACTIVE

능동 문구

use proactively, MUST BE
USED가 자동 위임을 강화.

SPECIFIC

구체 신호

auth, payments 변경 전처
럼 트리거 조건을 명시.

lowercase-
name 규칙

언제
description의 핵심

proactively
능동 위임 키워드

agent_type
혹 매처 값

description 작성 비교

위임되는 설명 vs 무시되는 설명

무시되는 설명 (막연함)

1. 코드를 리뷰하는 에이전트
2. 테스트 관련 도우미
3. 보안 전문가
4. 역할만 있고 시점이 없음
5. Claude가 위임 시점을 못 찾음

위임되는 설명 (시점 명확)

1. Use proactively after writing or modifying code
2. MUST BE USED when tests fail or coverage drops
3. Use before commits touching auth, payments, or user data
4. 행동 트리거가 문장에 내장
5. 매칭 조건이 곧 자동 위임 규칙

tools 필드 / 허용 목록

역할에 필요한 도구만 부여

tools allowlist

```
---  
name: safe-researcher  
description: Research agent with restricted capabilities  
tools: Read, Grep, Glob, Bash  
---
```

생략 시: 메인의 전체 도구를 상속 (MCP 포함)

지정 시: 나열한 도구만 사용 가능

Subagent에서 원천 불가한 도구

AskUserQuestion, EnterPlanMode, ScheduleWakeup,

WaitForMcpServers (UI, 세션 상태 의존)

Allowlist

tools 의미

생략=상속

MCP까지 전체

UI 도구

원천 사용 불가

Agent 포함

중첩 스폰 허용 스위치

disallowedTools와 MCP 패턴

차단 목록과 서버 단위 통제

denylist + mcp patterns

```
---
name: no-writes
description: Inherits every tool except file writes
disallowedTools: Write, Edit
---

# MCP 서버 단위 패턴
# mcp__github      github 서버의 전체 도구
# mcp__github__*   위와 동일 의미
# mcp__*           모든 MCP 도구 (deny 전용)

# 둘 다 지정 시: disallowed 먼저 제거 후 tools 해석
```

Denylist

상속에서 빠기

mcp__서버

서버 단위 매칭

mcp__*

MCP 전체 차단

deny 우선

동시 지정 해석 순서

model 필드

역할별 모델 배분

alias	sonnet, opus, haiku, fable	일반적인 선택
전체 ID	claude-opus-4-8, claude-sonnet-5	버전 고정 필요 시
inherit	메인 대화의 모델 그대로	생략 시 기본값
검증	조직 availableModels 허용 목록 통과 필수	제외 모델은 상속으로 폴백
사고 설정	확장 사고는 메인 설정을 그대로 상속	에이전트별 개별 설정 없음

inherit
생략 시 기본

fable
최상위 alias 지원

haiku
탐색 워커 권장

allowlist
조직 통제 적용

permissionMode 필드

에이전트별 권한 모드 오버라이드

default	표준 확인 프롬프트	일반 작업
acceptEdits	작업 경로의 편집 자동 수락	반복 수정 워커
auto	분류기가 명령과 보호 경로 쓰기 검토	신뢰 저장소
dontAsk	확인 프롬프트를 자동 거부	무인, 읽기 중심
plan	읽기 전용 탐색	조사 전용 워커
부모 우선	부모가 bypass, acceptEdits, auto면 그쪽 우선	프론트매터 무시됨

skills 프리로드

도메인 지식을 시작부터 주입

skills preload

```
---  
name: api-developer  
description: Implement API endpoints following team conventions  
skills:  
  - api-conventions  
  - error-handling-patterns  
---
```

```
# 나열한 스킬의 본문 전체가 시작 컨텍스트에 주입  
# 미나열 스킬도 Skill 도구로 실행 중 호출은 가능  
# 스킬 호출 자체를 막으려면 tools에서 Skill 제외
```

본문 전체

설명이 아닌 풀 주입

발견 불필요

탐색 단계 생략

Skill 도구

미나열분 온디맨드

역방향

skill의 context: fork

mcpServers 스코프

이 에이전트에게만 MCP 서버를

scoped mcp servers

```
---
name: browser-tester
description: Tests features in a real browser using Playwright
mcpServers:
  - playwright:          # 인라인: 이 에이전트 전용
    type: stdio
    command: npx
    args: ["-y", "@playwright/mcp@latest"]
  - github                # 참조: 기존 서버 공유
---
```

인라인 서버는 시작 시 연결, 종료 시 해제

인라인

에이전트 전용 연결

문자열

기존 서버 참조

컨텍스트 절약

메인에 정의 미노출

정책 적용

managed 제한 동일

memory 필드 / 영속 메모리

세션을 넘어 학습이 쌓이는 에이전트

● ● ● persistent memory

```
---  
name: code-reviewer  
description: Reviews code for quality and best practices  
memory: project  
---
```

You are a code reviewer. As you review code, update your agent memory with patterns, conventions, and recurring issues you discover.

```
# 스코프: user / project(권장) / local  
# MEMORY.md 첫 200줄 또는 25KB가 프롬프트에 포함
```

project

권장 기본 스코프

agent-memory/

저장 디렉토리

200줄/25KB

자동 로드 상한

R/W/E 자동

관리 도구 자동 부여

고급 필드 일람

실행 방식과 안전장치를 조율

<code>maxTurns</code>	에이전트 턴 수 상한	폭주 방지 안전장치
<code>effort</code>	low, medium, high, xhigh, max	세션 노력 수준 오버라이드
<code>isolation: worktree</code>	임시 git worktree에서 실행	변경을 체크아웃과 분리
<code>background: true</code>	항상 백그라운드로 실행	미지정 시 Claude가 선택
<code>color</code>	red, blue, green 등 8색	작업 목록 표시 색상
<code>initialPrompt</code>	--agent 메인 실행 시 첫 턴 자동 제출	세션형 에이전트 부팅

hooks / 프론트매터 후

이 에이전트가 도는 동안만의 검증

db-reader with PreToolUse hook

```
---
name: db-reader
description: Execute read-only database queries
tools: Bash
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-query.sh"
---
# 스크립트가 INSERT, UPDATE 등 감지 시 exit 2로 차단
```

수명 한정

에이전트 활성 중만

exit 2

차단 + 사유 회신

Stop 변환

SubagentStop으로

Plugin 무시

보안상 제외 필드

공식 예시 1 / code-reviewer

읽기 전용 리뷰어의 정석

● ● ● .claude/agents/code-reviewer.md

```
---
name: code-reviewer
description: Expert code review specialist. Proactively
  reviews code. Use immediately after writing or modifying code.
tools: Read, Grep, Glob, Bash
model: inherit
---
You are a senior code reviewer.
When invoked: 1. Run git diff 2. Focus on modified
files 3. Begin review immediately
Provide feedback by priority: Critical / Warnings /
Suggestions, with fix examples.
```

Edit 없음

읽기 전용 설계

즉시 착수

diff부터 실행

우선순위

3단계 출력 형식

Part 4

이 패턴 심화

공식 예시 2 / debugger

수정 권한을 가진 해결사

● ● ● .claude/agents/debugger.md

name: debugger

description: Debugging specialist for errors, test failures,
and unexpected behavior. Use proactively when
encountering any issues.

tools: Read, Edit, Bash, Grep, Glob

You are an expert debugger specializing in root cause
analysis.

When invoked: capture error and stack trace, isolate
the failure, implement minimal fix, verify solution.

Focus on fixing the underlying issue, not the symptoms.

Edit 포함

리뷰어와의 차이

최소 수정

행동 원칙 명문화

검증 포함

고치고 확인까지

증상 금지

근본 원인 지향

Part 2 Recap

정의 방법 핵심 정리

KEY TAKEAWAY

정의는 마크다운 한 장입니다.

description이 위임을 부르고, tools와 model이 역할을 조각하며, memory와 skills, 혹은 에이전트를 조직의 자산으로 만듭니다.

핵심 정리

1. 스코프 5계층, managed가 최상위
2. 정체성은 name, 위임은 description
3. tools 허용, disallowedTools 차단
4. memory: project로 학습 추적
5. 위저드 제거, Claude에게 생성 위임

다음 파트

1. Part 3 Agent 도구와 디스패치
2. 자동 위임과 명시 호출 3단계
3. 병렬, 체이닝, resume
4. fork와 중첩 실행
5. 관측과 디버깅

CHAPTER 02 / PART 03

Agent 도구와 디스패치

부르고, 병렬로 돌리고, 이어서 깨우기

1. 자동 위임과 명시 호출 3단계 사다리
2. 병렬 리서치, 고볼륨 격리, 체이닝
3. resume과 SendMessage, 중첩 실행
4. /fork 상세와 관측, 디버깅

Agent 도구 개요

Task에서 Agent로, 위임의 프리미티브

THE DELEGATION PRIMITIVE

메인 Claude는 Agent 도구를 호출해 서브에이전트를 만듭니다.

v2.1.63에서 Task 도구가 Agent로 개명되었고, 기존 Task(...) 표기는 별칭으로 계속 동작합니다.

RENAME

Task는 Agent

권한 규칙과 정의의 Task(...) 표기는 별칭으로 유효.

INPUT

위임 지시문

메인이 작업 요약과 함께 subagent_type을 지정해 호출.

OUTPUT

요약과 ID

완료 시 결과 요약과 agent ID가 메인으로 귀환.

DENY

차단 지점

Agent 도구 자체를 deny하면 모든 위임이 봉쇄.

v2.1.63

Task -> Agent 개명

Alias OK

기존 표기 호환

Agent(name)

권한 규칙 문법

깊이 5

중첩 상한 (후술)

자동 위임의 판단 재료

Claude는 무엇을 보고 넘기는가

AUTOMATIC DELEGATION

요청의 성격, 각 에이전트의 description, 현재 컨텍스트 세 가지를 종합해 위임을 결정합니다.
description의 능동 문구가 판을 기울입니다.

INPUT 1

요청 성격

사용자 프롬프트의 작업 서술

INPUT 2

description

각 에이전트의 시점 서술과 능동 문구

INPUT 3

컨텍스트

현재 대화와 작업 상태

DECIDE

위임 결정

매칭 에이전트에 지시문 작성 후 스폰

명시 호출 3단계 사다리

제안에서 보장, 세션 전체까지

ESCALATION LADDER

한 번의 제안은 자연어, 이번 작업의 보장은 @멘션, 세션 전체의 기본값은 --agent입니다. 강도가 한 칸씩 올라갑니다.

LEVEL 1

자연어

test-runner 서브에이전트로 고쳐줘,
Claude가 판단

LEVEL 2

@멘션

@agent-이름 지목,
이번 작업은 반드시 그 에이전트

LEVEL 3

--agent

세션 자체가 그 에이전트의 프롬프트와
도구로 실행

@멘션 상세

골뱅이로 에이전트를 지목

~/proj \$ claude

- > @ 입력 후 타입어헤드에서 선택
- > @"code-reviewer (agent)" auth 변경 부분 봐줘

수동 표기

로컬: @agent-code-reviewer

플러그인: @agent-my-plugin:review:security

타입어헤드에는 실행 중인 백그라운드

에이전트도 상태와 함께 표시

멘션은 어떤 에이전트가 돌지를 고정할 뿐,

지시문 작성은 여전히 메인 Claude의 몫

@agent-

수동 표기 접두

plugin:

스코프 이름 표기

상태 표시

실행 중 에이전트

보장

선택권을 고정

--agent 세션 전체 실행

메인 스레드가 에이전트가 된다

session as agent

```
$ claude --agent code-reviewer
```

```
# 세션 자체가 그 에이전트의 시스템 프롬프트,  
# 도구 제한, 모델로 실행
```

```
# 특성
```

```
# 기본 Claude Code 프롬프트를 완전 대체  
# CLAUDE.md와 프로젝트 메모리는 그대로 로드  
# 시작 헤더에 @이름 표시, resume 시에도 유지
```

```
# 프로젝트 기본값 고정 (.claude/settings.json)
```

```
# { "agent": "code-reviewer" }
```

```
# CLI 플래그가 설정보다 우선
```

--agent

세션 승격 플래그

프롬프트 대체

기본 CC 프롬프트 아님

메모리 유지

CLAUDE.md 정상 로드

agent 설정

프로젝트 기본값

패턴 / 병렬 리서치

독립 조사를 동시에

~/proj \$ claude

> 인증, 데이터베이스, API 모듈을 각각 별도 서브에이전트로 병렬 조사해줘. 각자 담당 영역의 구조와 핵심 흐름을 요약해서 보고해.

동작

독립 에이전트 3개가 동시에 탐색

각자 자기 컨텍스트에서 파일을 소화

완료되는 대로 요약이 메인에 도착

메인 Claude가 세 보고를 종합

조건: 조사 경로가 서로 의존하지 않을 것

동시 스폰

병렬 실행

독립 조건

의존 없는 경로

종합

메인이 합성

회수 주의

상세 결과 다발 비용

패턴 / 고볼륨 격리와 체이닝

대량 출력 소화와 순차 연결

~/proj \$ claude

고볼륨 격리

> 서브에이전트로 전체 테스트 스위트를 돌리고, 실패한 테스트와 에러 메시지만 보고해줘

수천 줄 로그는 자식에서 소화, 실패만 귀환

체이닝 (의존 작업의 순차 연결)

> code-reviewer 서브에이전트로 성능 이슈를 찾고,그다음 optimizer 서브에이전트로 고쳐줘

앞 결과를 메인이 받아 다음 지시문에 반영

격리

로그는 자식 소화

실패만

회수 형태 명시

체인

결과가 다음 입력

메인 중계

에이전트 간 직통 없음

백그라운드 운용

패널, 권한, 전환 키

BACKGROUND OPS

백그라운드 에이전트는 프롬프트 아래 패널에 나타납니다.

권한 요청은 메인에 떠오르고, 완료 결과는 메시지로 도착합니다.

PANEL

작업 패널

프롬프트 아래 행 단위 표시,
화살표로 이동, Enter로 열람.

PERMS

권한 중계

요청 에이전트 이름과 함께 승인
프롬프트가 메인에 표시.

STEER

중간 개입

열람 상태에서 후속 지시 전송,
방향 교정 가능.

KEYS

조작 키

Ctrl+B 백그라운드 전환, x 중지
와 정리, Esc 복귀.

Enter

트랜스크립트 열람

x

중지, 완료 정리

Ctrl+B

실행 중 전환

/tasks

작업 현황 명령

Resume과 SendMessage

종료된 에이전트를 이어서 깨우기

~/proj \$ claude

> code-reviewer 서버에이전트로 인증 모듈 리뷰해줘

완료, agent ID가 메인에 귀환

> 그 리뷰 이어서 이번엔 인가 로직을 분석해줘

Claude가 SendMessage로 해당 에이전트를 재개

이전 도구 호출과 추론을 전부 가진 채 계속

세부

중지된 에이전트는 메시지 수신 시 자동 재개

이름 재사용 충돌 시 전송 거부 후 대상 안내

Explore, Plan은 일회성이라 재개 불가

SendMessage

재개 수단 도구

전체 이력

이어지는 컨텍스트

자동 재개

중지 상태 수신 시

v2.1.199

이름 충돌 검사

중첩 서브에이전트

에이전트가 에이전트를 부린다

NESTED SPAWNING

v2.1.172부터 서브에이전트도 자기 서브에이전트를 만들 수 있습니다.

위임받은 작업이 다시 병렬 하위 작업으로 갈라질 때, 중간 출력이 메인에 닿지 않습니다.

DEPTH

깊이 5 상한

메인 아래 5단계까지, 고정 값이며 조정 불가.

ENABLE

활성 조건

해당 에이전트 tools에 Agent 포함 시 중첩 가능.

PANEL

트리 관측

패널 행에 하위 개수 표시, 열면 형제와 자식 경로.

RETURN

요약의 요약

최상위 서브에이전트의 요약만 메인으로 귀환.

v2.1.172+

중첩 지원 버전

5 levels

고정 깊이 상한

(+N)

패널 하위 표시

Fork 예외

fork는 fork 불가

/fork 상세

대화 전체를 물려받는 특수 서브에이전트

● ● ● ~/proj \$ claude

> /fork 지금까지의 파서 변경에 대한 단위 테스트 초안 작성

지시문 첫 단어들로 이름이 자동 부여

패널에 행 추가, 백그라운드로 진행

패널 조작

위아래 화살표 행 이동

Enter 트랜스크립트 열람, 후속 지시

x 중지 또는 완료 정리

Esc 프롬프트로 복귀

열람 중 /model, /fast는 메인 대상임을 안내

/fork

v2.1.161+ 기본 활성화

전체 상속

이력, 도구, 모델

캐시 공유

신규 스폰보다 저렴

worktree

격리 옵션 가능

Fork vs Named Subagent

무엇을 물려받는가의 차이

컨텍스트

Fork: 전체 대화 이력

Named: 지시문으로 새 출발

프롬프트, 도구

Fork: 메인과 동일

Named: 정의 파일의 것

모델

Fork: 메인과 동일

Named: model 필드

프롬프트 캐시

Fork: 메인과 공유

Named: 별도 캐시

선택 기준

Fork: 배경 설명이 긴 결가지

Named: 역할이 정형화된 작업

에러 처리

실패가 결과로 둔갑하지 않게

FAILURE SEMANTICS

v2.1.199부터 API 오류로 끊긴 에이전트는 오류 텍스트를 결과인 척 반환하지 않고 실패로 정확히 보고합니다.

부분 산출물은 함께 보존됩니다.

FG

포그라운드

부분 출력 + 중단 안내, 또는
명시적 종료 오류 반환.

BG

백그라운드

실패로 마킹, 종료 메시지에
오류명과 마지막 출력 포함.

RETRY

복구

원인 해소 후 재시도 요청 또는
해당 에이전트 resume.

GUARD

사전 안전장치

maxTurns 상한과 혹 검증으로
폭주와 위험 호출 차단.

v2.1.199

오류 의미론 개선

부분 보존

산출물 유실 방지

resume

이어서 복구

maxTurns

폭주 상한

관측과 디버깅

트랜스크립트, 압축 로그, 수명 주기 혹은

● ● ● observability

```
# 트랜스크립트 위치
~/claude/projects/{project}/{sessionId}/subagents/
agent-{agentId}.jsonl

# 압축 이벤트도 파일에 기록
{ "type": "system", "subtype": "compact_boundary",
  "compactMetadata": { "trigger": "auto",
                      "preTokens": 167189 } }

# settings.json의 수명 주기 혹은
# SubagentStart / SubagentStop, 이름 매치 지원
# 예: db-agent 시작 시 연결 준비, 종료 시 정리
```

JSONL

에이전트별 파일

preTokens

압축 직전 사용량

Start/Stop

수명 주기 이벤트

matcher

에이전트명 필터

선택 가이드 / Main vs Subagent vs /btw

상황별 최적 수단

찾은 왕복, 반복 다듬기	Main 대화	격리가 걸림돌
계획, 구현, 테스트가 맥락 공유	Main 대화	한 흐름 유지
대량 출력 작업, 요약만 필요	Subagent	격리의 본령
도구, 권한 제약을 강제할 작업	Subagent	역할 설계
대화 맥락에 대한 빠른 질문	/btw	전체 참조, 이력 미기록
재사용할 프롬프트, 워크플로	Skill	메인 컨텍스트에서 실행

Part 3 Recap

디스패치 핵심 정리

KEY TAKEAWAY

자연어, @멘션, --agent의 3단계로 부르고, 병렬과 체이닝으로 조합하며, ID와 SendMessage로 이어서 깨웁니다.
배경이 긴 결가지는 fork가 답입니다.

핵심 정리

- Task는 Agent로 개명, 별칭 유효
- 호출 사다리: 자연어, @멘션, --agent
- 패턴: 병렬, 격리, 체이닝
- 재개: agent ID + SendMessage
- 중첩 깊이 5, fork는 캐시 공유

다음 파트

- Part 4 Pattern 1 Code Reviewer
- 정의 완성본과 프롬프트 심화
- 영속 메모리 결합
- 대화형, 헤드리스, CI 세 경로
- 함정과 튜닝

CHAPTER 02 / PART 04

Pattern 1 / Code Reviewer

가장 수요가 많은 첫 상비군

1. 시나리오와 정의 완성본
2. 시스템 프롬프트 심화와 영속 메모리 결합
3. 대화형, 헤드리스, GitHub Actions 세 경로
4. 확장 변형과 자주 만나는 함정

시나리오 정의

무엇을 자동화하려는가

THE SCENARIO

커밋 전 셀프 리뷰를 표준화합니다.

사람 리뷰어에게 가기 전에 명백한 결함과 보안 이슈를 걸러, 리뷰 왕복 횟수를 줄이는 것이 목표입니다.

TRIGGER

발동 시점

코드 작성이나 수정 직후, 커밋과 PR 생성 전.

SCOPE

검토 범위

git diff의 변경 파일 중심,
저장소 전수 검사 아님.

OUTPUT

산출 형식

치명, 경고, 제안 3단계와 파일, 라인, 수정 예시.

SAFETY

안전 원칙

읽기 전용, 코드를 직접 고치지 않고 보고만.

Pre-commit

핵심 발동 지점

diff 중심

검토 범위

3단계

우선순위 출력

Read-only

권한 원칙

정의 완성본

시나리오를 파일로 옮기면

● ● ● .claude/agents/code-reviewer.md

```
---
name: code-reviewer
description: Expert code review specialist. Proactively
  reviews code. Use immediately after writing or modifying code.
tools: Read, Grep, Glob, Bash
model: inherit
memory: project
---

You are a senior code reviewer ensuring high standards
of code quality and security.
When invoked: run git diff, focus on modified files,
begin review immediately.
```

No Edit

읽기 전용 강제

proactively

자동 위임 문구

memory

project 스코프

Bash 포함

git diff 실행용

시스템 프롬프트 심화

체크리스트와 출력 형식을 본문에

system prompt body

Review checklist:

- 명확성: 함수와 변수 이름, 중복 코드
- 안전성: 에러 처리, 입력 검증, 시크릿 노출
- 품질: 테스트 커버리지, 성능 고려

Provide feedback organized by priority:

- Critical (must fix): 파일:라인 + 수정 예시
- Warnings (should fix)
- Suggestions (consider improving)

결론에 머지 가능 여부를 한 줄로 판정.

체크리스트

검토 항목 고정

파일:라인

위치 특정 강제

수정 예시

실행 가능한 피드백

판정 한 줄

머지 가능 여부

영속 메모리 결합

리뷰할수록 똑똑해지는 리뷰어

LEARNING REVIEWER

memory: project로 이 저장소의 반복 이슈와 컨벤션이 축적됩니다.

팀은 메모리 디렉토리를 버전 관리에 올려 학습까지 공유합니다.

WRITE

축적 지시

본문에 발견 패턴을 메모리에 기록하라는 지시 포함.

READ

선참조 지시

리뷰 전에 과거 패턴을 먼저 확인하라고 호출 시 요청.

SHARE

팀 공유

.claude/agent-memory/
커밋으로 학습 자체를 공유.

CURATE

정리 주기

MEMORY.md 상한 초과 시
에이전트가 스스로 큐레이션.

project

권장 스코프

반복 이슈

축적 대상

커밋 가능

학습의 팀 자산화

자가 정리

상한 초과 시

호출 경로 1 / 대화형

자동 위임과 멘션

● ● ● ~/proj \$ claude

자동 위임 (description 매칭)

> 결제 모듈 리팩토링 끝났어, 커밋 전에 점검하자

수정 직후 문맥이 code-reviewer를 자동 발동

멘션으로 보장

> @agent-code-reviewer 이번 diff 봐줘.

메모리의 과거 패턴 먼저 확인하고 시작해.

체이닝

> 리뷰에서 Critical만 debugger 서브에이전트로 바로 수정하고 재검토까지 돌려줘

자동

문맥 발동

@멘션

실행 보장

선참조

메모리 활용 지시

체인

리뷰 후 수정 연결

호출 경로 2 / 헤드리스

스크립트와 후에서

headless invocation

단발 실행

```
claude -p "code-reviewer 서브에이전트로 현재 diff를 리뷰하고 Critical 개수를 마지막 줄에 N건 형식으로"
```

pre-push 훅 게이트 (예시)

```
RESULT=$(claude -p "...위 지시..." \
  --allowed-tools "Agent,Read,Grep,Glob,Bash(git diff:*)")
echo "$RESULT" | tail -1 | grep -q "0건" || {
  echo "Critical 발견, push 중단"; exit 1; }
```

-p
비대화 실행

Agent 허용
위임 도구 포함 필수

형식 지시
마지막 줄 판정 규칙

exit 1
게이트 차단

호출 경로 3 / GitHub Actions

PR마다 자동 리뷰

● ● ● .github/workflows/agent-review.yml

```
on: [pull_request]
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4 # with fetch-depth: 0
      - run: curl -fsSL https://claude.ai/install.sh | bash
      - run: |
          claude -p "code-reviewer 서브에이전트로 이 PR
            diff 리뷰, 결과를 review.md로 저장" \
            --allowed-tools "Agent,Read,Grep,Glob,Bash,Write"
    env: { ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY } }
```

PR 트리거

pull_request 이벤트

정의 동봉

저장소의 agents 사용

review.md

코멘트 게시 소스

Secrets

키는 시크릿 주입

내장 /code-review 와의 관계

언제 명령을, 언제 내 에이전트를

COMPLEMENTARY

둘은 대체가 아니라 보완 관계입니다.

즉석 점검은 내장 명령, 팀 기준의 상비군은 맞춤 에이전트입니다.

/code-review (내장 명령)

1. 설치 즉시 사용, 정의 불필요
2. fix, comment, ultra 옵션 내장
3. 일반 기준의 범용 리뷰
4. 즉석 점검과 마무리 정리에 적합

code-reviewer (맞춤 에이전트)

1. 팀 체크리스트와 출력 형식 고정
2. 영속 메모리로 저장소 학습 추적
3. 혹, CI, 체이닝에 이름으로 참여
4. 리뷰 기준의 버전 관리와 개선

확장 변형

리뷰어 패밀리로 분화

VAR 1

security-reviewer

인증, 결제 경로 전담, 커밋 전 필수 발동
문구.

VAR 2

perf-reviewer

N+1, 캐시, 복잡도 등 성능 관점 전담.

VAR 3

api-reviewer

계약 호환성, 버저닝, 문서 일치 검토.

VAR 4

test-reviewer

테스트 품질과 커버리지 갭 전담.

VAR 5

nested 구성

리뷰어가 발견 항목별 검증 워크를 중첩
스폰.

VAR 6

opus 상향

아키텍처 판단이 무거운 저장소는
model: opus.

자주 만나는 함정

현장에서 걸리는 지점

함정

1. description에 시점이 없어 자동 위임 불발
2. Bash 누락으로 git diff 실행 불가
3. 출력 형식 미지정, 리뷰가 산문으로 흩어짐
4. 저장소 전수 검사로 시간과 토큰 폭증
5. CI에서 Agent 도구 미허용으로 위임 실패

솔루션

1. Use immediately after... 시점 문구 명시
2. tools에 Bash 포함, 혹은 diff만 허용
3. 우선순위 3단계와 파일:라인 형식 강제
4. diff 중심 범위를 본문에 명문화
5. allowed-tools에 Agent 포함 확인

Part 4 Recap

Code Reviewer 핵심 정리

KEY TAKEAWAY

읽기 전용 + 시점 있는 설명 + 형식화된 출력 + 영속 메모리, 이 네 가지가 팀 리뷰어의 공식입니다.

세 호출 경로로 어디서든 같은 기준이 됩니다.

핵심 정리

1. 시나리오가 프론트매터를 결정
2. 체크리스트와 형식을 본문에 고정
3. memory: project로 학습 축적
4. 대화형, 헤드리스, CI 세 경로
5. 내장 명령과는 보완 관계

다음 파트

1. Part 5 Pattern 2 Tester
2. test-writer 정의
3. 커버리지 확장 워크플로
4. worktree 격리 실행
5. CI 통합

CHAPTER 02 / PART 05

Pattern 2 / Tester

커버리지를 넓히는 실행형 워커

1. test-writer 정의와 커버리지 워크플로
2. 엣지 케이스 자동 발굴
3. worktree 격리로 안전한 실행
4. 프레임워크 대응과 CI 통합

test-writer 정의

쓰고 실행하고 고치는 워커

●●● .claude/agents/test-writer.md

```
---
name: test-writer
description: Test coverage specialist. Use proactively
  when new code lacks tests or coverage drops.
tools: Read, Write, Edit, Bash, Grep, Glob
model: sonnet
isolation: worktree
maxTurns: 40
---
You are a test automation expert. Workflow: measure
coverage, find gaps, write tests matching project
conventions, run them, fix failures preserving intent.
```

Write 포함

테스트 파일 생성

worktree

격리 실행

maxTurns 40

폭주 상한

의도 보존

실패 수정 원칙

커버리지 확장 워크플로

측정에서 통과까지 5단계

STEP 1

측정

커버리지 리포트
실행, 현황 파악

STEP 2

갭 선정

미커버 분기와 위험
모듈 우선순위

STEP 3

작성

프로젝트 컨벤션으
로 케이스 작성

STEP 4

실행

신규 포함 전체 실행
, 실패 수집

STEP 5

수정

의도 보존하며 통과
까지 반복

각 단계의 산출물이 다음 단계의 입력, 최종 보고는 커버리지 변화와 신규 케이스 목록

리포트 먼저

감이 아닌 측정

위험 우선

결제, 인증 모듈

컨벤션

기존 스타일 준수

Before/After

보고 형식

엣지 케이스 자동 발굴

사람이 놓치는 경계를 긁어내기

EDGE MINING

코드를 읽으며 분기와 예외 경로에서 케이스 후보를 추출합니다.

발굴 관점을 본문에 명시하면 놓치는 축이 줄어듭니다.

AXIS 1

경계값

0, 음수, 최대치, 빈 문자열,
빈 배열.

AXIS 2

예외 경로

타임아웃, 재시도, 부분 실패
, 예외 전파.

AXIS 3

동시성

중복 요청, 경합, 순서 역전.

AXIS 4

시간과 로컬

시간대, 월말, 윤년, 다국어
입력.

분기 추적

발굴의 출발점

4축

본문 명시 관점

위험 태그

케이스별 근거 주석

사람 검토

후보 승인 흐름

실전 대화 예시

위임에서 보고까지

~/proj \$ claude

> @agent-test-writer src/payments의 커버리지를 80% 이상으로 올려줘. 만료 카드와 부분 환불 경로를 반드시 포함하고, 끝나면
전후 수치로 보고해.

백그라운드 패널에서 진행 관찰

완료 보고 (요약 귀환)

커버리지 62% -> 84%

신규 12 케이스: 만료 카드 3, 부분 환불 4,

경계값 5 / 전체 스위트 통과

> 그중 부분 환불 케이스들 diff 보여줘

목표 수치

완료 기준 명시

필수 경로

도메인 지식 주입

전후 보고

형식 지시

후속 검토

diff 확인 흐름

프레임워크와 Mock 전략

생태계별 대응

JS/TS

Jest, Vitest, Playwright 자동 인지

기존 설정 파일 우선

Python

pytest, fixture와 parametrize 활용

conftest 컨벤션 준수

Java/Kotlin

JUnit 5, Mockito

빌드 도구 명령 준수

Mock 원칙

외부 IO만 mock, 내부 로직은 실물

과도 mock은 검증력 훼손

픽스처

기존 팩토리과 헬퍼 재사용 우선

중복 생성 금지 지시

worktree 격리 실행

내 체크아웃을 건드리지 않는 실험실

ISOLATION: WORKTREE

테스터는 임시 git worktree에서 작업합니다.

대량 파일 생성과 실행이 내 작업 트리와 분리되고, 변경이 없으면 워크트리는 자동 정리됩니다.

BASE

분기 기준

기본 브랜치에서 분기, 내 미 커밋 변경과 무관.

SAFE

충돌 없음

내가 편집 중인 파일과 절대 겹치지 않음.

REVIEW

회수 방식

결과를 diff나 브랜치로 검토 후 선택적 병합.

CLEAN

자동 정리

무변경 종료 시 워크트리 자동 삭제.

1줄 설정

isolation: worktree

병렬 안전

테스터 다중 실행

기본 브랜치

분기 기준점

/batch 원리

동일 격리 기술

CI 통합

커버리지 하락을 자동 방어

● ● ● .github/workflows/coverage-guard.yml

```
on: { schedule: [{ cron: "0 21 * * 0" }] } # 일 06시 KST
jobs:
  expand:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: curl -fsSL https://claude.ai/install.sh | bash
      - run: |
          claude -p "test-writer 서버에이전트로 커버리지
            최하위 모듈 1개를 보강하고 PR 브랜치 생성" \
            --allowed-tools "Agent,Read,Write,Edit,Bash,Grep,Glob"
    env: { ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY } }
```

주간 크론
정기 보강

1 모듈
회당 범위 제한

PR 산출
사람 리뷰 게이트

Routines 대안
관리형 예약 경로

테스트 품질 지표

숫자 너머의 건강도

METRIC 1

커버리지 추세

절대값보다 하락 감지,
모듈별 최저선 관리.

METRIC 2

의도 보존율

수정 시 assertion 약화
여부를 리뷰에서 점검.

METRIC 3

플레이키 비율

간헐 실패 테스트 격리와
원인 추적.

METRIC 4

실행 시간

스위트 소요 추세, 병렬화
와 분할 시점 판단.

추세

절대값보다 방향

Assertion

약화 감시 항목

Flaky

격리 대상

회수 보고

지표를 요약에 포함

Part 5 Recap

Tester 핵심 정리

KEY TAKEAWAY

쓰기 권한에는 격리를 짝지웁니다.

worktree와 maxTurns, 의도 보존 원칙이 실행형 워커의 3대 안전장치입니다.

핵심 정리

1. 실행형: Write, Edit, Bash 부여
2. 5단계: 측정, 갭, 작성, 실행, 수정
3. 엣지 4축을 본문에 명시
4. worktree 격리 + 턴 상한
5. 완료 기준은 수치로

다음 파트

1. Part 6 Pattern 3 Security Scanner
2. 읽기 전용 + 혹 검증 조합
3. 시크릿과 의존성 스캔
4. pre-commit 게이트
5. 오탐 처리와 메모리

CHAPTER 02 / PART 06

Pattern 3 / Security Scanner

읽기 전용과 혹 검증의 결합

1. security-scanner 정의와 탐지 능력
2. 시크릿과 의존성 스캔
3. pre-commit 게이트와 dontAsk 무인 운용
4. 오탐 처리와 메모리 추적, 컴플라이언스

security-scanner 정의

읽기 전용 + PreToolUse 이중 잠금

.claude/agents/security-scanner.md

```
---
name: security-scanner
description: Security specialist. MUST BE USED before
  commits touching auth, payments, or user data.
tools: Read, Grep, Glob, Bash
model: opus
memory: project
permissionMode: dontAsk
hooks: { PreToolUse: [{ matcher: "Bash", hooks:
  [{ type: command, command: "./scripts/ro-guard.sh" }] }] }
---
```

MUST BE USED

강제 위임 문구

opus

판단 난도 상향

dontAsk

무인 자동 거부

ro-guard

Bash 읽기 전용 검증

ro-guard.sh / 혹은 검증 스크립트

읽기 명령만 통과시키는 문지기

./scripts/ro-guard.sh

```
#!/bin/bash
INPUT=$(cat)
CMD=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

# 허용: 조회 계열만
echo "$CMD" | grep -qE \
    '^(git (diff|log|show|status)|grep|rg|cat|head|ls|find|npm audit|pip-audit|trivy)' \
    && exit 0

echo "Blocked: read-only commands only" >&2
exit 2
```

stdin JSON

혹 입력 형식

allowlist

조회 계열만 통과

exit 2

차단 + 사유 회신

audit 도구

스캐너 명령 포함

취약점 탐지 능력

본문에 명시하는 6대 관점

SCAN 1

인젝션

SQL, 커맨드, 템플릿 인젝션과 이스케이프 누락.

SCAN 2

인증, 인가

검증 우회 경로, 권한 상승, 세션 고정.

SCAN 3

민감 데이터

시크릿 하드코딩, 로그 노출, 평문 저장.

SCAN 4

입력 검증

경계 미검증, 역직렬화, 경로 조작.

SCAN 5

설정 결함

CORS 과개방, 디버그 모드, 기본 자격증명

SCAN 6

의존성

알려진 CVE, 유지보수 중단 패키지.

실전 대화 예시

결제 모듈 커밋 전 점검

~/proj \$ claude

- > 결제 웹훅 핸들러 수정했어, 커밋 전에 점검하자
 - # auth, payments 문맥 -> security-scanner 자동 발동
 - # 완료 보고 (요약 귀환)
 - # Critical 1: 웹훅 서명 검증이 타임스탬프 미확인,
 - # 재전송 공격 가능 (src/webhooks/pay.ts:42)
 - # Warning 2: 에러 응답에 내부 스택 노출 외 1건
 - # Info: npm audit high 1건 (fast-xml-parser)
- > Critical부터 debugger 서브에이전트로 수정하고
재스캔까지 체이닝해줘

자동 발동

민감 문맥 매칭

파일:라인

위치 특정 보고

공격 시나리오

왜 위험한지 설명

체이닝

수정과 재스캔 연결

시크릿과 의존성 스캔

코드 밖의 두 전선

BEYOND CODE

하드코딩된 자격증명과 취약 의존성은 코드 로직과 별개의 전선입니다.

스캐너 본문에 두 절차를 상비 항목으로 넣습니다.

시크릿 탐지

1. API 키, 토큰, 비밀번호 패턴 검색
2. 엔트로피 높은 문자열 후보 추출
3. .env 예시 파일과 실파일 구분
4. 이력 유출 시 회전 절차 안내까지

의존성 스캔

1. npm audit, pip-audit 실행과 해석
2. 심각도별 분류, 수정 버전 확인
3. 직접, 전이 의존 구분 보고
4. 락파일 기준 재현 가능한 결과

pre-commit 게이트 통합

치명 이슈는 커밋 자체를 차단

.git/hooks/pre-commit

```
#!/bin/bash
STAGED=$(git diff --cached --name-only | grep -E \
  'auth|payment|user' || true)
[ -z "$STAGED" ] && exit 0 # 민감 경로 없으면 통과

RESULT=$(claude -p "security-scanner 서브에이전트로
  스테이징된 변경을 스캔, 마지막 줄에 Critical N건" \
  --allowed-tools "Agent,Read,Grep,Glob,Bash")

echo "$RESULT" | tail -1 | grep -q "Critical 0건" || {
  echo "보안 Critical 발견, 커밋 중단"; exit 1; }
```

경로 필터

민감 변경만 발동

판정 규격

마지막 줄 N건

exit 1

커밋 차단

우회 금지 합의

no-verify 정책화

오탐 처리와 메모리

늑대 소년이 되지 않는 법

FALSE POSITIVE LOOP

오탐이 반복되면 게이트는 무시됩니다.

확인된 오탐을 메모리에 축적해 같은 지적이 되풀이되지 않게 만드는 순환이 핵심입니다.

TRIAGE

판정 흐름

발견 항목을 사람이 확정,
오탐은 사유와 함께 기록.

MEMORY

축적 지시

확인된 오탐 패턴을 메모리에
저장하라고 본문에 명시.

RECHECK

선참조

스캔 전 메모리의 오탐 목록을
먼저 대조.

EXPIRE

재검토 주기

오탐 기록에 근거 링크와 날짜
, 분기별 재검토.

사유 필수

오탐 기록 형식

선대조

스캔 전 참조

분기 재검토

기록 만료 관리

정확도 상승

회차 누적 효과

컴플라이언스 연계

감사 대응을 부산물로

증적 생성

스캔 보고를 날짜별 파일로 보존

SubagentStop 후 자동화

표준 매핑

발견 항목에 OWASP 분류 태깅

본문 출력 형식에 포함

범위 증명

민감 경로 목록과 발동 조건 문서화

게이트 스크립트가 근거

예외 관리

오탐, 수용 위험의 사유와 승인자

메모리 기록이 대장

정기 점검

주간 전체 스캔 크론

CI 크론 또는 Routines

Part 6 Recap

Security Scanner 핵심 정리

KEY TAKEAWAY

도구 허용 목록과 혹 검증의 이중 잠금이 무인 보안 게이트를 가능하게 합니다.

오탐 메모리 순환이 게이트의 신뢰를 지킵니다.

핵심 정리

1. 이중 잠금: tools + PreToolUse 혹
2. dontAsk로 무인 운용
3. 탐지 6관점을 본문에 명시
4. pre-commit 게이트, 판정 규칙
5. 오탐은 메모리로 순환 관리

다음 파트

1. Part 7 Pattern 4 Docs Writer
2. 문서 생성 전담 워커
3. README, API 문서, ADR
4. skills 프리로드로 스타일 준수
5. Changelog와 CI

CHAPTER 02 / PART 07

Pattern 4 / Docs Writer

코드를 따라가는 문서 전담반

1. docs-writer 정의와 skills 프리로드
2. README, API 문서, ADR 생성
3. Changelog 자동화
4. CI로 문서 부패 방지

docs-writer 정의

skills 프리로드가 스타일을 고정

.claude/agents/docs-writer.md

name: docs-writer

description: Documentation specialist. Use proactively
when public APIs change or docs drift from code.

tools: Read, Write, Edit, Grep, Glob, Bash

model: sonnet

skills: [docs-style-guide, api-doc-template]

You are a technical writer. Read the code first,
document what it actually does, and follow the
preloaded style guide exactly.

skills 2종

스타일, 템플릿 주입

코드 우선

실제 동작 기준

Write 포함

문서 파일 생성

drift 문구

과리 감지 시 발동

README 자동 생성

저장소의 첫인상 표준화

~/proj \$ claude

> @agent-docs-writer 이 저장소 README를 표준 구조로 재작성해줘. 실제 package.json 스크립트와 .env.example 기준으로 설치, 실행 절차를 검증해서.

산출 구조 (스킬 템플릿 기준)

개요와 아키텍처 한 단락

Quick Start: 검증된 설치, 실행 명령

구성: 환경변수 표

개발: 테스트, 린트, 배포 절차

명령은 실제 실행해 확인 후 기재

실검증

명령 실행 후 기재

env 표

구성 문서화

템플릿

스킬이 구조 고정

첫인상

온보딩 시간 단축

API 문서 생성

코드에서 계약을 추출

~/proj \$ claude

> @agent-docs-writer src/api/routes를 스캔해서 엔드포인트 문서를 docs/api.md로 만들어줘.
요청, 응답 스키마는 타입 정의에서 추출하고 에러 코드 표까지 포함해.

엔드포인트별 산출 항목
메서드, 경로, 설명, 인증 요건
요청 파라미터와 바디 스키마
응답 예시 (성공, 실패)
에러 코드와 의미 표

타입 추출
스키마의 근거

에러 표
실패 계약 명시

예시 쌍
성공과 실패

OpenAPI
확장 산출 선택지

ADR 작성

결정의 이유를 남기는 문서

● ● ● ~/proj \$ claude

> 세션 저장소를 Redis로 바꾸기로 했어.
@agent-docs-writer 이번 대화와 diff를 근거로 ADR 초안 작성해줘.

```
# ADR 골격 (스킬 템플릿)
# Status: Proposed / Accepted / Superseded
# Context: 문제와 제약
# Decision: 선택과 근거
# Alternatives: 검토했으나 기각한 안과 이유
# Consequences: 트레이드오프와 후속 작업
```

결정 직후
작성 타이밍

기각안 포함
재논쟁 방지

번호 연번
docs/adr/ 관리

후임자
왜의 보존 대상

Changelog 자동 생성

커밋 이력을 사용자 언어로

~/proj \$ claude

> @agent-docs-writer v2.3.0 태그 이후 커밋들로 CHANGELOG 항목 작성해줘. 사용자 관점 문장으로, Breaking은 마이그레이션 안 내까지.

분류 규칙

Added / Changed / Fixed / Deprecated / Removed

Breaking Changes는 최상단, 대응 방법 필수

내부 리팩토링은 제외

conventional commits면 분류 정확도 상승

태그 기준

범위 산정

사용자 문장

커밋 메시지 번역

Breaking 상단

마이그레이션 포함

릴리스 후

자동화 연결 지점

skills 프리로드 심화

스타일 준수의 메커니즘

STYLE BY PRELOAD

스타일 가이드 스킬의 본문 전체가 시작 컨텍스트에 주입되므로, 에이전트는 규칙을 찾는 것이 아니라 이미 아는 상태로 출발합니다.

WHAT

주입 내용

어투, 용어집, 금지 표현, 섹션 구조, 예시 문단.

VS MEMORY

메모리와 분업

스킬은 팀 규범, 메모리는 이 저장소의 발견 사항.

UPDATE

개정 반영

스킬 파일 수정이 다음 호출 부터 즉시 적용.

SCOPE

이 워커만

메인 대화 컨텍스트는 소비하지 않음.

본문 전체

주입 단위

규범 vs 발견

스킬, 메모리 분업

즉시 개정

파일 수정 반영

0 부담

메인 컨텍스트

CI / 문서 부패 방지

API 변경과 문서를 동기화

● ● ● .github/workflows/docs-sync.yml

```
on: { pull_request: { paths: ["src/api/**"] } }
jobs:
  docs:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: curl -fsSL https://claude.ai/install.sh | bash
      - run: |
          claude -p "docs-writer 서버에이전트로 이 PR의 API
            변경을 docs/api.md에 반영, 변경 없으면 무동작" \
            --allowed-tools "Agent,Read,Write,Edit,Grep,Glob,Bash"
    env: { ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY } }
```

paths 필터

API 변경만 발동

무동작 조건

불필요 커밋 방지

동일 PR

코드와 문서 동행

부패 차단

괴리 원천 방지

Part 7 Recap

Docs Writer 핵심 정리

KEY TAKEAWAY

코드가 근거, 스킬이 스타일, CI가 동기화를 맡으면 문서 부패가 구조적으로 차단됩니다.

핵심 정리

1. skills 프리로드로 스타일 고정
2. 명령은 실검증 후 기재
3. API 문서는 타입에서 추출
4. ADR은 기각안까지 기록
5. paths 필터 CI로 동기화

다음 파트

1. Part 8 Pattern 5 Migration Bot
2. 대규모 일괄 변경 워커
3. 시나리오 3종
4. 안전장치 3종 구성
5. /batch와의 관계

CHAPTER 02 / PART 08

Pattern 5 / Migration Bot

대규모 일괄 변경의 안전 설계

1. migration-bot 정의와 안전장치 3중 구성
2. 라이브러리 업그레이드, import 경로, deprecated API
3. 모델 배분으로 비용 최적화
4. /batch와의 관계

migration-bot 정의

안전장치 3종의 실행 워커

.claude/agents/migration-bot.md

```
---
name: migration-bot
description: Large-scale migration specialist. Use for
  library upgrades, import paths, deprecated API swaps.
tools: Read, Write, Edit, Bash, Grep, Glob
model: sonnet
isolation: worktree
maxTurns: 60
memory: project
---

Migrate in small verifiable batches: run build and tests
each batch, record progress to memory, stop on threshold.
```

worktree

안전장치 1 격리

maxTurns 60

안전장치 2 상한

배치 검증

안전장치 3 절차

memory

진행 상황 기록

시나리오 1 / 라이브러리 업그레이드

메이저 버전 전환

● ● ● ~/proj \$ claude

> @agent-migration-bot lodash 4를 제거하고 네이티브 ES 메서드로 전환해줘. 대체 불가 유틸은 src/utils/에 자체 구현하고, 10파일 단위로 빌드와 테스트 검증하면서 진행해.

진행 보고 (배치마다 메모리 기록)

batch 3/9: 28 files done, tests green

치환 불가 2건: debounce, cloneDeep -> utils 구현

완료 후 worktree diff를 브랜치로 회수

10파일 배치

검증 단위

대체 규칙

불가분 처리 방침

메모리

중단 후 재개 근거

브랜치 회수

PR 리뷰 흐름

시나리오 2 / Import 경로 변경

모듈 재배포와 배럴 정리

~/proj \$ claude

> @agent-migration-bot src/common을 packages/shared로 옮기는 중이야. 전체 import 경로를 갱신하고 배럴 파일 재수출도 정리 해줘. tsconfig paths 별칭 기준으로, 상대 경로 신규 생성은 금지.

검증 체인

tsc --noEmit -> lint -> test

세 관문 통과 배치만 다음으로

순환 참조 발견 시 목록화 후 중단, 사람 판단 요청

별칭 기준

경로 규칙 고정

3관문

타입, 린트, 테스트

순환 중단

구조 문제는 사람에게

배럴 정리

재수출 일관성

시나리오 3 / Deprecated API 교체

폐기 예정 호출 일소

~/proj \$ claude

> @agent-migration-bot 사내 SDK의 fetchUserV1 계열이 다음 분기 폐기돼. v2 API로 전량 교체해줘.
시그니처 차이는 마이그레이션 가이드 문서 기준, 의미가 바뀌는 5개 케이스는 목록만 만들어.

산출

기계 교체: 143 call sites, 전량 테스트 통과

판단 필요: 5건 목록 + 각 사유 (동작 의미 변화)

회귀 방지: v1 호출 재유입 차단 lint 규칙 추가

가이드 기준

치환 규칙 근거

판단 분리

의미 변화는 목록만

lint 규칙

재유입 차단

143 sites

기계 교체 규모

안전장치 종합

대규모 변경의 4중 방어

DEFENSE IN DEPTH

격리, 상한, 절차, 회수의 네 겹입니다.

어느 하나가 뚫려도 다음 겹이 피해를 막는 구조를 대규모 변경의 기본값으로 삼으세요.

LAYER 1

worktree 격리

내 체크아웃과 분리, 무변경 시 자동 정리.

LAYER 2

상한과 차단기

maxTurns 상한, 실패율 초과 시 자진 중단.

LAYER 3

배치 검증 절차

빌드, 테스트 관문을 배치마다 통과.

LAYER 4

리뷰 회수

브랜치와 PR로만 본선 합류, 사람 승인 게이트.

4겹

격리, 상한, 절차, 회수

자진 중단

실패율 서킷 브레이커

PR only

본선 합류 경로

재개 가능

메모리 진행 기록

비용 효율 / 모델 배분

탐색은 싸게, 변환은 정확하게

COST SPLIT

전수 탐색과 실제 변환의 난도가 다릅니다.

단계별로 모델을 나누면 품질 저하 없이 비용이 크게 내려갑니다.

탐색 단계 (haiku 워커)

1. 대상 호출부 전수 수집
2. 파일별 영향도 분류
3. 치환 가능, 판단 필요 선별
4. 산출: 작업 목록과 배치 계획

변환 단계 (sonnet 본대)

1. 배치 계획대로 정밀 치환
2. 관문 검증과 실패 수습
3. 판단 필요 건 사유 정리
4. 산출: 브랜치와 완료 보고

/batch와의 관계

내 봇과 내장 배치의 분업

BOT VS /BATCH

batch는 조사, 분해, 병렬 스폰, 단위별 PR까지 자동인 내장 오케스트레이터입니다.

맞춤 봇은 그 워커로도, 단독으로도 쓰입니다.

/batch (내장 오케스트레이션)

1. 코드베이스 조사와 단위 분해 자동
2. 5에서 30개 워커 병렬 스폰
3. 워크트리 격리와 단위별 PR
4. 대규모 군질 변경에 최적

migration-bot 단독 운용

1. 배치 크기, 관문, 규칙을 세밀 통제
2. 메모리로 회차 간 연속성
3. 판단 분리 등 도메인 규칙 내장
4. 중간 규모, 반복 마이그레이션에 최적

Part 8 Recap

Migration Bot 핵심 정리

KEY TAKEAWAY

대규모 변경은 격리, 상한, 절차, 회수의 4겹 방어가 기본값입니다.

기계 치환과 사람 판단의 경계를 지시에 명시하는 것이 품질을 가릅니다.

핵심 정리

1. 안전장치 4겹 방어
2. 시나리오 3종의 공통 골격
3. 판단 필요 건은 목록으로 분리
4. haiku 탐색 + sonnet 변환 분업
5. 대규모 균질 변경은 /batch 우선

다음 파트

1. Part 9 Recap & Labs
2. 챕터 핵심 요약
3. 실습 랩 4종
4. FAQ와 학습 자료
5. Q&A와 다음 챕터 예고

CHAPTER 02 / PART 09

Recap & Labs

챕터 정리와 실습 4종

1. Chapter 2 핵심 요약과 체크리스트
2. FAQ와 추가 학습 자료
3. 실습 랩 4종
4. Q&A와 다음 챕터 예고

Chapter 2 핵심 요약

아홉 파트를 여섯 문장으로

CONCEPT

개념

격리 컨텍스트에서 일하고 요약만 돌려주는 세션 내 워커.

DEFINE

정의

Markdown 한 장, 필수는 name과 description, 스코프 5계층.

FIELDS

필드

tools, model, memory, skills, hooks, isolation의 조합 설계.

DISPATCH

디스패치

자연어, @멘션, --agent 사다리와 병렬, 체이닝, resume.

ADVANCED

고급

fork의 전체 상속과 캐시 공유, 중첩 깊이 5, SendMessage.

PATTERNS

패턴 5종

리뷰어, 테스터, 스캐너, 문서, 마이그레이션의 검증된 골격.

학습 목표 체크리스트

네 가지 목표 자가 점검

SELF CHECK

아래 네 문장에 답할 수 있다면 목표 달성입니다.
막히는 항목은 표기된 파트로 돌아가 복습하세요.

CHECK 1

설명할 수 있다

격리 모델과 fork, Teams
의 경계를 설명. (Part 1, 3)

CHECK 2

정의할 수 있다

프론트매터로 역할별 에이전트
작성. (Part 2)

CHECK 3

제어할 수 있다

멘션, 병렬, 체이닝,
resume 운용. (Part 3)

CHECK 4

구축할 수 있다

5패턴 중 둘 이상을 우리
저장소에 배포. (Part 4-8)

4 Checks

목표 점검 항목

Part 참조

복습 경로 표기

Labs

미완 항목은 실습으로

배포 기준

팀 저장소 커밋

FAQ / 자주 나오는 질문

현장 질문 여섯 가지

Subagent끼리 대화가 되나요

불가, 메인인 항상 중계

필요하면 Agent Teams

정의를 고치면 재시작해야 하나요

파일 위치가 수 초 내 반영

새 폴더 첫 생성만 예외

/agents 위저드가 안 보여요

v2.1.198에서 제거됨

Claude에게 생성 위임

비용이 두 배로 들지 않나요

haiku 배분과 fork 캐시로 통제

Part 1, 8 비용 슬라이드

에이전트가 폭주하면요

maxTurns, 혹, worktree 격리

Part 5, 8 안전장치

팀 표준은 어떻게 배포하나요

.claude/agents 커밋 + managed

Part 2 스코프 계층

Lab 1 / 첫 Subagent 만들기

소요 15분

● ● ● LAB 1 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 데크와 인증 환경만 준비하시면 됩니다
```

15분

예상 소요

생성 위임

공식 생성 경로

@멘션

즉시 호출 확인

위치 체험

무재시작 반영

Lab 2 / 도구 권한 격리

소요 15분

● ● ● LAB 2 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 데크와 인증 환경만 준비하시면 됩니다
```

15분

예상 소요

거동 관찰

권한 밖 요청 반응

denylist

상속에서 빼기

exit 2

혹 차단 확인

Lab 3 / 디스패치와 병렬

소요 20분

● ● ● LAB 3 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 데크와 인증 환경만 준비하시면 됩니다
```

20분

예상 소요

패널

관측 조작 체득

3 병렬

동시 스폰 체험

체인

결과 연결 흐름

Lab 4 / 메모리와 Fork

소요 25분

● ● ● LAB 4 STEPS

```
# 본 랩은 워크샵 웹사이트에서 진행합니다
$ open https://github.com/whchoi98/claude-code-workshop
  -> ccw-hands-on-lab / 해당 챕터의 랩 문서 선택
# 사전 준비물, 단계별 절차, 검증 체크리스트가 웹 문서로 제공됩니다
# 로컬에는 이 데크와 인증 환경만 준비하시면 됩니다
```

25분

예상 소요

2회 순환

기록과 선참조

MEMORY.md

축적 확인 지점

/fork

전체 맥락 상속

추가 학습 자료

이 챕터 다음에 읽을 것들

Subagents 공식 문서

code.claude.com/docs/ko/sub-agents

본 텍스트의 1차 출처

Agent Teams

docs 내 agent-teams 문서

세션 간 협업 확장

Hooks 레퍼런스

docs 내 hooks 문서

검증 스크립트 심화

Skills

docs 내 skills 문서

프리로드 대상 제작

Worktrees

docs 내 worktrees 문서

격리 실행 원리

워크샵 코드

github.com/whchoi98/claude-code-workshop

본 과정 실습 자료

Q&A

질문과 토론

DISCUSSION

정의 설계, 팀 배포 전략, 비용 통제 무엇이든 좋습니다.

시간상 못 다룬 질문은 워크샵 저장소 이슈로 남겨주시면 답변드리겠습니다.

TOPIC 1

설계

역할 분리 기준, 프론트매터 조합 관련 질문.

TOPIC 2

운영

무인 게이트, CI 통합, 오탐 관리 관련 질문.

TOPIC 3

확장

Teams 전환 시점, 대규모 병렬 관련 질문.

Next / Chapter 3 예고

Admin Setup

CHAPTER 3 PREVIEW

개인의 도구를 조직의 플랫폼으로 올립니다.

대규모 배포와 자격증명, 사내망, 거버넌스가 다음 챕터의 주제입니다.

NEXT 1

배포 전략

조직 단위 설치, 버전 통제,
managed settings.

NEXT 2

자격증명

SSO, Bedrock IAM, 키 볼
트와 헬퍼 설계.

NEXT 3

사내망

프록시, 도메인 허용, 미러와
에어갭 대응.

NEXT 4

거버넌스

정책 계층, 감사 로그, OTel
비용 관측.

Ch.3

Admin Setup

Enterprise

관점 전환

Day 2

워크샵 일정

To be continued

다음 세션에서

References

본 챕터 공식 문서 출처 (code.claude.com)

Sub-agents	docs/ko/sub-agents	Part 1, 2, 3 전반
Agent Teams	docs/ko/agent-teams	Part 1 경계 지도
Hooks	docs/ko/hooks	Part 2, 6 혹 검증
Skills	docs/ko/skills	Part 2, 7 프리로드
Worktrees	docs/ko/worktrees	Part 5, 8 격리
Model config	docs/ko/model-config	Part 1 모델 해석

Thank you!

Choi, WooHyung / Prin. Solutions Architect, AWS Korea

whchoi@amazon.com | linkedin.com/in/woohyungchoi

Workshop code: github.com/whchoi98/claude-code-workshop